

III Year I semester

**SOFTWARE AUTOMATION AND
TESTING**

Unit-IV

UNIT IV

What is Testing, Characteristics of Test Engineers, Software Testing Life Cycle, Levels of Testing, Testing Approaches, Test Cases: Format for Writing Test Case, Test plan: Format to prepare Test plan Purpose of testing, Dichotomies, Consequences of bugs

Definitions:

“ Testing is the process to prove that the software works correctly”

This definition sounds good, but note that this is not a good definition of testing process. If the person who has developed the software carries out the testing , will try to use this definition.

The person who developed the software will only show that the software works correctly so he will give only those inputs for which correct results are obtained.

“ Testing is the process to prove that the software does not work”.

Strictly, if the aim of the test engineer is to prove that the software does not work , then the testing process can be considered good.

The SQA team and even the user/client teams try to use this definition while carrying out software testing.

“ Testing is the process of executing the program with the intention of finding errors” .

“ Testing is the process to detect the defects and minimize the risk associated with the residual defects”

The test engineers need to understand the application well, study the software functionality in detail to find out where the bugs are likely to occur, study the code to ensure that each and every line of code is tested, create test cases in such a way that testing is done rigorously to uncover the hidden bugs and also ensure that the software is usable and reliable.

Characteristics:

❑ **Understand the application domain:** The software may be client/server based database management system, or a telecom software or data communication software or embedded software and so on. A test engineer have to test software of various application domains.

❑ Before starting the testing, you need to understand the application, from an end user point of view.

❑ **Learn about the development environment:** Based on the application and the end user needs, the development environment varies- the os may be windows or unix or linux , the programming language may be c , C++ or java etc etc

❑ Learn how to use the application: The test engineer has to assume that he is the end user and run the application to get feel of how the software works, only then he will be in position to generate the test cases and also create automated test procedures.

❑ Study the source code: If the test engineer asked to validate the source code also, he need to study the source code based on which the test case generation has to be done.

❑ Study the requirement specification document: The SRS document gives the functional, performance and reliability requirements of the software.

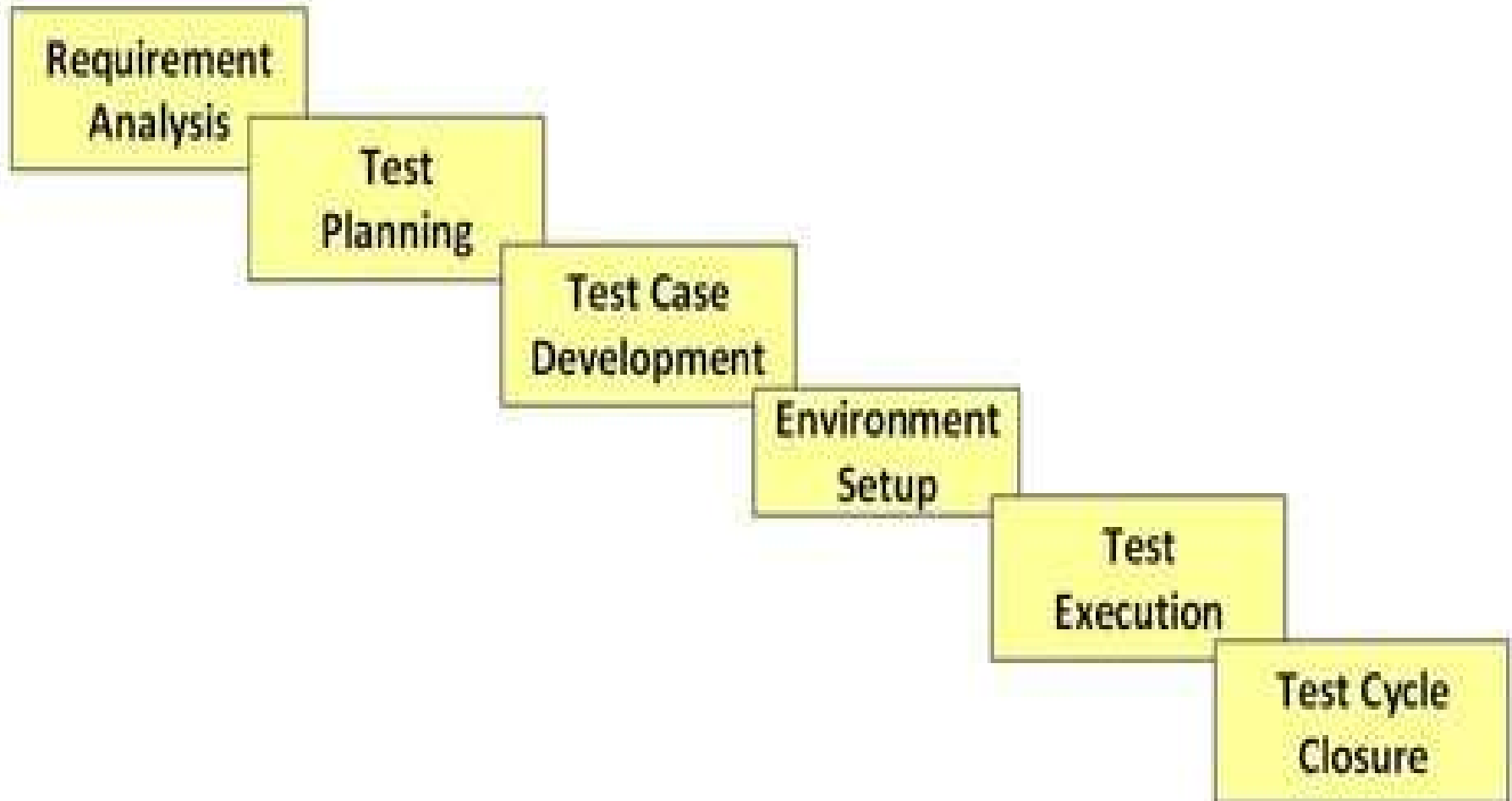
- ❑ the software has to be tested for all these requirements.

- ❑ You need to be eagle eyed and look for detail while studying the specifications and check whether the requirements can be tested or not.

- ❑ If SRS document says, the transaction response should be very fast- it does not mean anything? You should tell the person who wrote the specification to indicate the value say 10 seconds. Then only you can test performance test. Metrics are very important.

- ❑ **Study the acceptance test procedure:** The ATP document gives the test plan based on which the user will validate the software.
 - ❑ The test engineers need to carry out the testing as per the ATP document.
 - ❑ The ATP document has to be prepared by the development team and the testing team in association with the customer representative.

The different stages in Software Test Life Cycle :



Requirement Analysis:

- ✓ During this phase, test team studies the requirements from a testing point of view to identify the testable requirements.
- ✓ The QA team may interact with various stakeholders (Client, Business Analyst, Technical Leads, System Architects etc) to understand the requirements in detail.
- ✓ Requirements could be either Functional (defining what the software must do) or Non Functional (defining system performance /security availability) .
- ✓ Automation feasibility for the given testing project is also done in this stage.

Activities

- Identify types of tests to be performed.
- Gather details about testing priorities and focus.
- Prepare [Requirement Traceability Matrix \(RTM\)](#).
- Identify test environment details where testing is supposed to be carried out.
- Automation feasibility analysis (if required).

Deliverables

- RTM
- Automation feasibility report. (if applicable)

Test Planning

- ✓ This phase is also called **Test Strategy** phase.
- ✓ Typically , in this stage, a Senior QA manager will determine effort and cost estimates for the project and would prepare and finalize the Test Plan.

Activities

- Preparation of test plan/strategy document for various types of testing
- Test tool selection
- Test effort estimation
- Resource planning and determining roles and responsibilities.
- Training requirement

Deliverables

- Test plan /strategy document.
- Effort estimation document

Test Case Development

- ✓ This phase involves creation, verification and rework of test cases & test scripts.
- ✓ **Test data** , is identified/created and is reviewed and then reworked as well.

Activities

- Create test cases, automation scripts (if applicable)
- Review and baseline test cases and scripts
- Create test data(If Test Environment is available)

Deliverables

- Test cases/scripts
- Test data

Test Environment Setup

- ✓ Test environment decides the software and hardware conditions under which a work product is tested.
- ✓ Test environment set-up is one of the critical aspects of testing process and *can be done in parallel with Test Case Development Stage*.
- ✓ *Test team may not be involved in this activity* if the customer/development team provides the test environment in which case the test team is required to do a readiness check ([smoke testing](#)) of the given environment.

Activities

- Understand the required architecture, environment set-up and prepare hardware and software requirement list for the Test Environment.
- Setup test Environment and test data
- Perform smoke test on the build

Deliverables

- Environment ready with test data set up
- Smoke Test Results.

Test Execution

- ✓ During this phase test team will carry out the testing based on the test plans and the test cases prepared.
- ✓ Bugs will be reported back to the development team for correction and retesting will be performed.

Activities

- Execute tests as per plan
- Document test results, and log defects for failed cases
- Map defects to test cases in RTM
- Retest the defect fixes
- Track the defects to closure

Deliverables

- Completed RTM with execution status
- Test cases updated with results
- Defect reports

Test Cycle Closure

✓ Testing team will meet , discuss and analyze testing artifacts to identify strategies that have to be implemented in future, taking lessons from the current test cycle. The idea is to remove the process bottlenecks for future test cycles and share best practices for any similar projects in future.

Activities

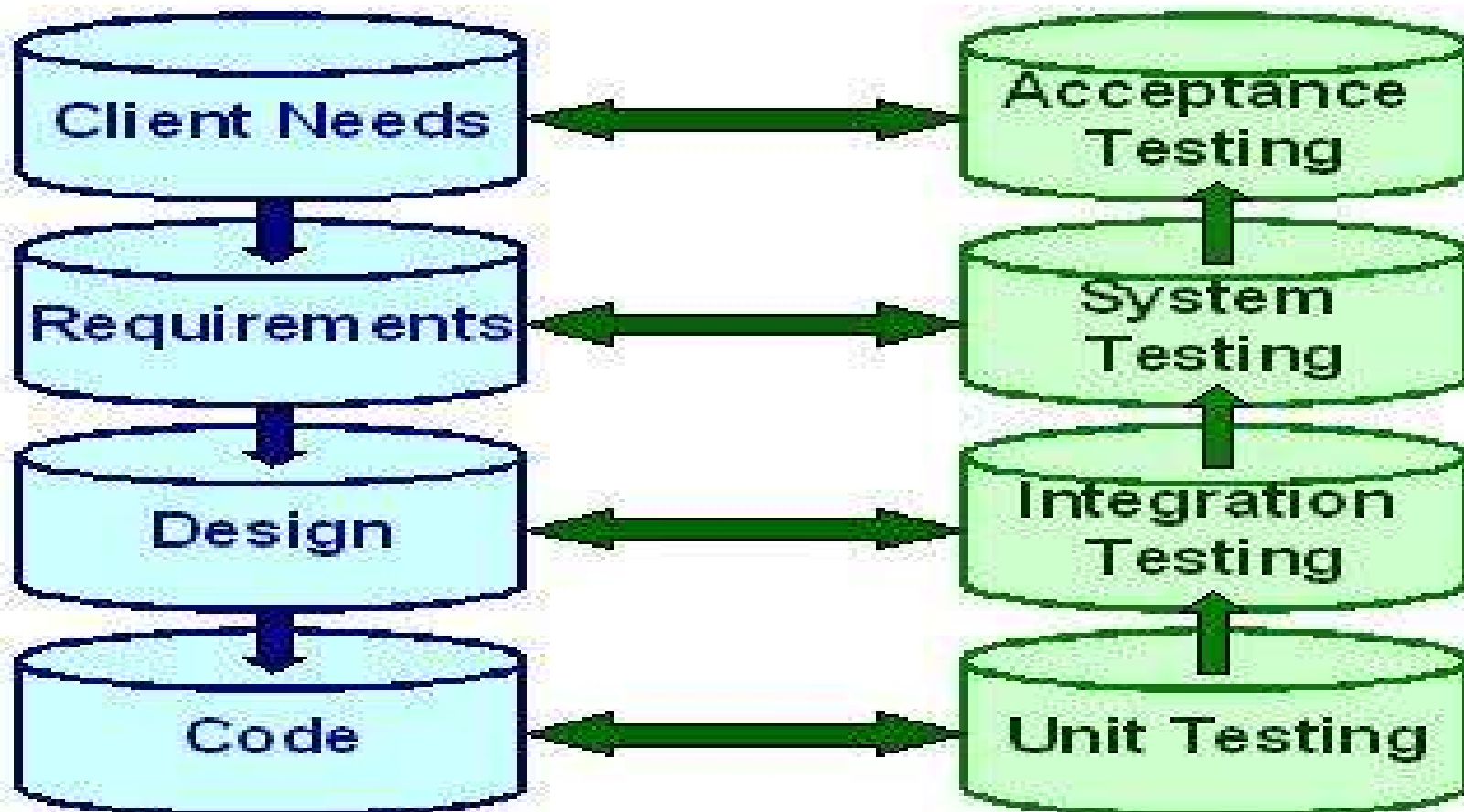
- Evaluate cycle completion criteria based on Time, Test coverage, Cost, Software, Critical Business Objectives , Quality
- Prepare test metrics based on the above parameters.
- Document the learning out of the project
- Prepare Test closure report
- Qualitative and quantitative reporting of quality of the work product to the customer.
- Test result analysis to find out the defect distribution by type and severity.

Deliverables

- Test Closure report
- Test metrics

- ❖ During the design stage, the software is divided into modules and each module is divided into units.
- ❖ While testing it is no good if one combines all the units into modules and all the modules into the system and then starts testing the entire system.
- ❖ A practical approach is to divide the testing process into different levels. To start with, each unit has to be tested separately and then the modules have to be built from the units and the module are tested, then the modules are combined together and the system is built and tested.
- ❖ unit testing is done to test the source code, integration testing is done to test design and system testing is done to test SRS, and finally acceptance testing is done to test the client/user requirements.

Pictorial Representation of Levels of Testing



www.softwaretestinggenius.com

Unit Testing:

- ✓ A unit is the smallest entity in the software
- ✓ Every unit has to be tested separately to check whether it is as per the specifications
- ✓ As it is not possible to test a unit individually, additional piece of code may need to be written to test the units.
- ✓ unit testing is normally done by the development engineers themselves.

Module Testing:

- ✓ A Module is an independent entity in the software
- ✓ The tested units are integrated into a module and each module is tested separately for the specifications.
- ✓ For instance, if the software is a database management system, the modules can be data base(back end) and user interface (front end).
- ✓ The database can be tested separately using the SQL commands to check the correct data retrieval.
- ✓ After testing the module, a module level test report has to be prepared. Once every module is working as per requirements, the next phase of testing will start i.e integration and system testing.

Integration and system testing:

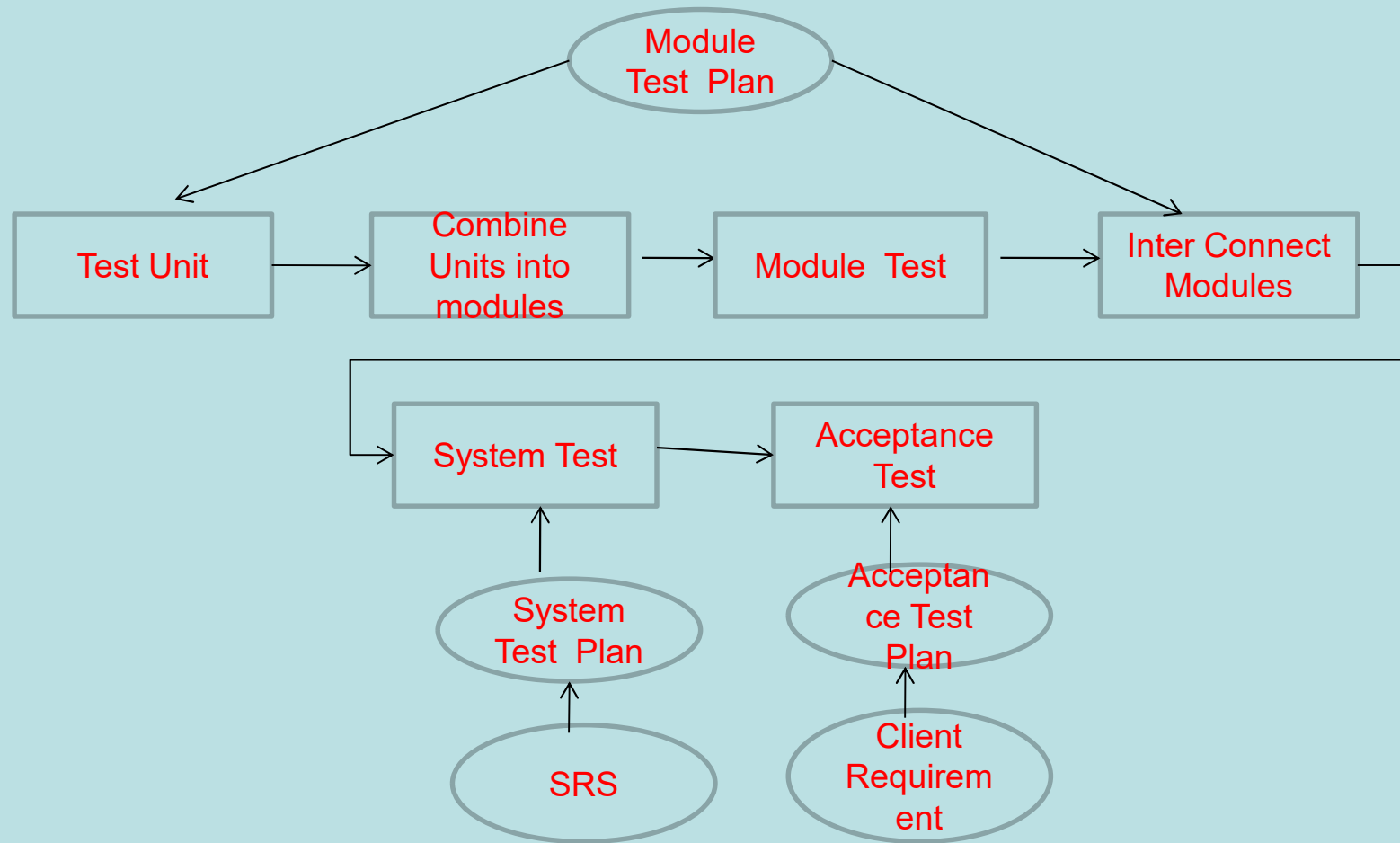
- ✓ After the modules are tested, the modules can be integrated together .
- ✓ It is difficult to integrate all the modules together and start testing as it would be difficult to do debugging.
- ✓ The integration has to be done systematically by incremental building and testing in steps.
- ✓ Once all the modules are integrated together, system testing is carried out for functional and non-functional requirements.

Acceptance Testing:

- ✓ After the system testing is completed, the software is tested by the client/user. This is known as acceptance testing.
- ✓ Acceptance testing can be done either at the developer's premises or client premises.

Levels of Testing

L1



- ❖ In a large software project, it is impractical to integrate all the modules in one shot and start testing the software as a whole.
- ❖ The system has to be built in stages and the product has to be built incrementally
- ❖ Such a systematic approach helps in easier debugging

Approaches:

Top-down approach and Bottom-Up approach:

- ❑ **Top-Down Approach:** In this approach, testing is done from the top of the hierarchy.
- ❑ **Bottom-Up Approach:** In this approach, testing is done from the bottom of the hierarchy

Functional Testing and Structural Testing:

- ❑ In Functional testing, the functionality of the module is tested and the structure is not considered

- ❑ Test cases based on specifications and internals of modules are not considered. This type of testing is known as **Black box testing**
- ❑ Structural testing is used to test the implementation of the program, here the source code is looked into for testing the software.
- ❑ Structural testing involves (a) Statement coverage (b) branch coverage (c) Path coverage

Mutation Testing:

- ❑ Mutation testing is required to ensure that the software does not fail.
- ❑ It is a good debugging mechanism
- ❑ After the software works correctly, mutation testing can be done to simulate wrong inputs
- ❑ in mutation testing, program is modified (or logic is changed) slightly to obtain mutants of the program
- ❑ Different mutants are tested with the same test cases
- ❑ if the mutants fails, and the actual program works correctly, confidence is gained in the program and test cases are considered as good i.e different mutants should give different results

- ❑ generally each mutant will have only one change
- ❑ To produce mutants, mutation operators are defined, mutant operators can be
 - ❑ Constant replacement
 - ❑ Variable replacement
 - ❑ Arithmetic operator replacement
 - ❑ Relational operator replacement
 - ❑ Goto label replacement

Ex:

Regression Testing:

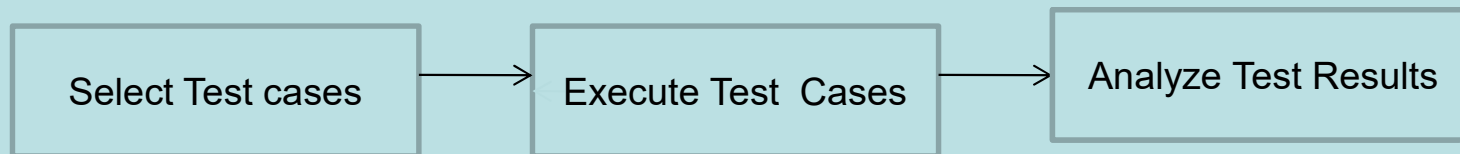
- ❑ Testing a Modified build is called a Regression testing.
- ❑ When a defect is reported in the software, the developer makes some changes to the software to remove the defect.
- ❑ It is likely that the change made in the code may lead to another defect that may not be visible immediately so whenever a change is made to the source code, one has to ensure that there are no ill effects of the change on the other parts of the software.
- ❑ whenever a change is made to the source code, a set of pre-defined test cases has to be run to check whether any other portion of the software is affected.

Ex: Consider the following code

```
Input x;  
y=x**2;  
print y;  
if(y==100)  
print "Alarm: x value is 10";
```

- Initially, you wrote the program which takes a number as input, squares the number and check whether the square of the number is 100 or not.
- If it is 100, it prints an alarm.
- Suppose late on, you change the logic and calculate the value of y as cube of x instead of square of x
 - You may simply change the second line as $y=x*x*x$;
- Suppose you now test your program by giving some inputs 2,6, and 8 and find that program works, strictly it wont work.
 - Because you need to change the if statement

- ❑ Test cases can be defined as sets of inputs parameters for which the software will be tested.
- ❑ Below fig shows the Testing process using test cases



- ❑ Strictly all combinations of inputs must be checked. If this is done, it is known as exhaustive testing.
- ❑ However, exhaustive testing is costly, time consuming and impossible in many cases, so it is highly impractical. So a subset of all combinations is used for testing- this is known as test cases.

❑ **Test case definition:**

A test case is step by step instructions to test a specific requirement

Or

Step by step driving direction to check specific functionality.

- one test case can contain one or more steps based on the complexity of requirement.
- Simple and clear steps or driving directions to test the s/w functionality.

➤ **Test Scenario:** *A document specifying a sequence of actions for the execution of a test.*

➤ **Test Data:** *Data that exists before a test is executed, and that affects or is affected by the component or system under test.*

- ❑ Test cases have to be designed based on two criteria : reliability and validity.
- ❑ A set of test cases is considered to be reliable, if it detects all errors
- ❑ A set of test cases is considered as valid if at least one test case reveals the errors

Techniques used for selecting the Test Cases:

- **Equivalence class partitioning** : In this technique all possible valid and valid outputs are identified
- The test cases consists of both valid inputs and invalid inputs
- **Boundary Value Analysis**: Values that are on the boundary of the equivalence classes are high yield test cases.
- **Cause Effect Graphing**: In this approach for each cause(inputs) and effects (outputs) is identified and a cause effect graph is drawn.

How to create Effective test case:

1. Create test case based on requirements:

- Business requirements are the input for test case. Define test conditions based on detailed requirements
- Develop multiple cases to test individual business requirements when necessary.

2. Validate Test Case:

- Validate the test cases are defined correctly to test requirements and that cases provide adequate coverage of requirements
- Review with business Functional expert
- It is not always possible to test everything . cases should target high risk functions first for more extensive testing.

3. Maintain Traceability:

- Map test cases to requirements, components and scripts
- This ensure that the test documents are maintainable

1. Traceability is very important when the requirements or test conditions change and it becomes necessary to update the test \

2. scripts Repeability:

- Manual scripts need to be detailed enough for everyone to follow cases

1. Manual and Automated:

2. Target high risk functions:

- Test cases may be manual or automated

IEEE format for test case:

- 1. Test Case Id:** Unique Name or ID.
- 2. Test Case Name:** Name of the test condition
- 3. Features To Be Tested:** Corresponding Module/ Function/ Service
- 4. Test Suite Id:** In which batch, is this case is a member of
- 5. Priority:** Importance of the test case in terms of functionality
 - P0: Basic Functionality
 - P1: General Functionality (Input Domain, Error Handling, Compatibility)
 - P2: Cosmetic Testing (User Interface)
- 6. Test Environment:** Required Hardware and Software to execute this test case
- 7. Test Effort (Person/Hour):** Time required to execute this test case (E.g. 20 minutes on an average)

8. Test Duration: Date and Time (Scheduling)

9. Test Setup: Necessary tasks to do before starting this test case execution

10. Test Procedure: A step-by-step process to execute these test cases from base state to end state

Serial No.	Action	Input Required	Expected	Actuals	Result	Comments

Filled during test design stage filled during test execution

11. Test Case Pass/Fail Criteria: when this test case is pass and when this test case fails

❖ Before taking up the testing, a test plan has to be prepared.

Def: Test Plan is a systematic approach to testing typically contain detailed information of what the work load will be. It deals with detailed information about upcoming testing efforts, scope of testing, test deliverables, staff and risk etc.

Template: A template is a file that serves as a starting point for a new document, when we open a template, it is pre-formatted in some way

Document: A piece of written printed or electronic matter that provides information or evidence or that serves as an official record

❖ Test plan template contain following template:

1. Test plan identifier:

- ✓ It is a uniquely generated company identifier to identify the test plan.
- ✓ Test plan identifier may be used to identify whether the testplan is a master plan or level plan or integration plan

2) Introduction:

- ✓ Describe the purpose of the plan, possibly identifying the level of the plan(system test plan etc..).
- ✓ This is essentially the executive summary part of the plan

3) Test items:

- ✓ There are things you intend to test within the scope of this test plan

4)References:

- ✓ List all documents that support this test plan. Refer to the actual version/Release number of the document as stored in the configuration management system

5) Features to be tested:

- ✓ This is a listing of what is to be tested from the users viewpoint of what the system does.
- ✓ This is not a technical description of the software, but a users view of the functions

6) Features not to be Tested:

- ✓ This is a listing of what is Not to be tested from both users viewpoint of what the system does and a configuration management/version control view. This is not a technical description of the software, but a users view of the functions

7) Test Approach:

- ✓ This is your overall test strategy for this test plan. It should be appropriate to the level of the plan(master,acceptance,etc) and should be in agreement with all higher and lower levels of plans. Overall rules and processes should be identified.

8) Entry Criteria:

- ✓ It describes when to start testing

9) Exit Criteria:

- ✓ It describes when to stop testing

10) Suspension criteria

- ✓ It describes when to stop testing temporarily

11) Roles and Responsibilities:

- ✓ Team leader or test lead and team members roles and responsibilities

12) Schedule:

- ✓ Schedule for all test activities in this software test process

13) Training:

- ✓ Training on the application /system (DomainTesting)
- ✓ Training for any test tools to be used

14) Test Environment/lab

- ✓ It describes required hardware and software for setting –up test environment or test lab

15) Test Deliverables:

- ✓ List out that what is to be delivered as part of this plan

16) Approvals

- ✓ Who can approve the process as complete and allow the project to proceed to the next level

17) Glossary

- ✓ Define terms and acronyms used in the document, it can be used to understand the terms used in this plan.

Test Plan ID: Unique No. or Id or Name of the test plan

Introduction: About the Project and testing

Test Items: Names of Modules/ Functions/ Services/ Features

Features To Be Tested: Responsible Modules for the Test Design

Features Not To Be Tested: Which ones to test and which ones not to test (e.g. Features of previous version of the Software)

Approach: List of testing techniques to be applied on the modules (prepared by QA/PM)

Features Pass/Fail Criteria: When above features are pass and when they fail

Suspension Criteria: Possible abnormal situations arose during testing of above features. Without recovering from these situations, you are not able to conduct testing. (Technical problems with respect to project)

Test Environment: Required hardware and software including testing tools to conduct testing

Test Deliverables: Required test documents to be prepared during testing (Test Cases, Test Procedures, Test Log, Test Report)

Test Tasks: necessary tasks to do before starting of every project testing

Staff and Training Needs: The names of test engineers and required training sessions

Purpose of Testing

1. To Catch Bugs

- Bugs are due to imperfect Communication among programmers
 - Specs, design, low level functionality
- Statistics say: about 3 bugs / 100 statements

2. Productivity Related Reasons

- Insufficient effort in QA => High Rejection Ratio => Higher Rework => Higher Net Costs
- Statistics:
 - QA costs: 2% for consumer products
80% for critical software
- Quality ⇔ Productivity

Purpose of Testing

Purpose of testing contd...

3. Goals for testing

Primary goal of Testing: Bug Prevention

- * Bug prevented $\Rightarrow$ rework effort is saved [bug reporting, debugging, correction, retesting]
- * If it is not possible, Testing must reach its secondary goal of bug discovery.
- * Good test design & tests $\Rightarrow$ clear diagnosis $\Rightarrow$ easy bug correction

Test Design Thinking

- * From the specs, write test specs. First and then code.
- * Eliminates bugs at every stage of SDLC.
- * If this fails, testing is to detect the remaining bugs.

4. 5 Phases in tester's thinking

Phase 0: says no difference between debugging & testing

- Today, it's a barrier to good testing & quality software.

Purpose of Testing

Purpose of testing contd...

Phase 1: says Testing is to show that the software works

- A failed test shows software does not work, even if many tests pass.
- Objective not achievable.

Phase 2: says Software does not work

- One failed test proves that.
- Tests are to be redesigned to test corrected software.
- But we do not know when to stop testing.

Phase 3: says Test for Risk Reduction

- We apply principles of statistical quality control.
- Our perception of the software quality changes – when a test passes/fails.
- Consequently, perception of product Risk reduces.
- Release the product when the Risk is under a predetermined limit.

Purpose of Testing

5 Phases in tester's thinking continued...

Phase 4: A state of mind regarding “What testing can do & cannot do. What makes software testable”.

- Applying this knowledge reduces amount of testing.
- Testable software reduces effort
- Testable software has less bugs than the code hard to test

Cumulative goal of all these phases:

- Cumulative and complementary. One leads to the other.
- Phase2 tests alone will not show software works
- Use of statistical methods to test design to achieve good testing at acceptable risks.
- Most testable software must be debugged, must work, must be hard to break.

Purpose of Testing

purpose of testing contd..

5. Testing & Inspection

- * Inspection is also called static testing.
- * Methods and Purposes of testing and inspection are different, but the objective is to catch & prevent different kinds of bugs.
- * To prevent and catch most of the bugs, we must
 - * Review
 - * Inspect &
 - * Read the code
 - * Do walkthroughs on the code

& then do Testing

Dichotomies

Dichotomies

- * division into two especially mutually exclusive or contradictory groups or entities
- * the *dichotomy* between theory and practice

Let us look at six of them:

1. Testing & Debugging
2. Functional Vs Structural Testing
3. Designer vs Tester
4. Modularity (Design) vs Efficiency
5. Programming in **SMALL** Vs programming in **BIG**
6. Buyer **vs** Builder

Dichotomies

1. Testing Vs Debugging

- Testing is to find bugs.
- Debugging is to find the cause or misconception leading to the bug.
- Their roles are confused to be the same. But, there are differences in goals, methods and psychology applied to these

#	Testing	Debugging
1	Starts with known conditions. Uses predefined procedure. Has predictable outcomes.	Starts with possibly unknown initial conditions. End cannot be predicted.
2	Planned, Designed and Scheduled.	Procedures & Duration are not constrained.
3	A demo of an error or apparent correctness.	A Deductive process.
4	Proves programmer's success or failure.	It is programmer's Vindication.
5	Should be predictable, dull, constrained, rigid & inhuman.	There are intuitive leaps, conjectures, experimentation & freedom.

Dichotomies

Dichotomies contd...

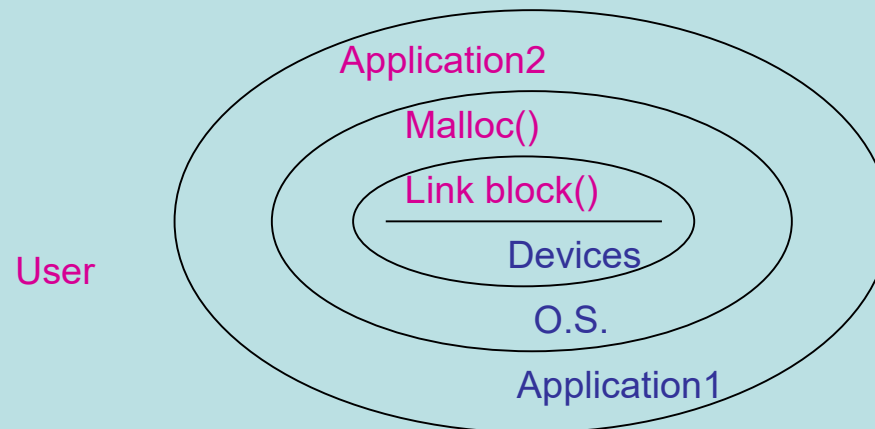
#	Testing	Debugging
6	Much of testing can be without design knowledge.	Impossible without a detailed design knowledge.
7	Can be done by outsider to the development team.	Must be done by an insider (development team).
8	A theory establishes what testing can do or cannot do.	There are only Rudimentary Results (on how much can be done. Time, effort, how etc. depends on human ability).
9	Test execution and design can be automated.	Debugging - Automation is a dream.

Dichotomies

Dichotomies contd..

2. Functional Vs Structural Testing

- * **Functional Testing:** Treats a program as a **black box**. Outputs are verified for **conformance** to specifications from user's point of view.
- * **Structural Testing:** Looks at the implementation details: **programming style, control method, source language, database & coding details.**
- * **Interleaving of functional & Structural testing:**
 - * A good program is built in layers from outside.
 - * Outside layer is pure system function from user's point of view.
 - * Each layer is a structure with its outer layer being its function.
 - * Examples:



Dichotomies

- * Interleaving of functional & Structural testing: (contd..)
 - * For a given model of programs, Structural tests may be done first and later the Functional, Or vice-versa. Choice depends on which seems to be the natural choice.
 - * Both are useful, have limitations and target different kind of bugs. Functional tests can detect all bugs in principle, but would take infinite amount of time. Structural tests are inherently finite, but cannot detect all bugs.
 - * The Art of Testing is how much allocation % for structural vs how much % for functional.

Dichotomies

Dichotomies contd..

3. Designer vs Tester

- ✓ Completely separated in black box testing. Unit testing may be done by either.
- ✓ Artistry of testing is to balance knowledge of design and its biases against ignorance & inefficiencies.
- ✓ Tests are more efficient if the designer, programmer & tester are independent in all of unit, unit integration, component, component integration, system, formal system feature testing.
- ✓ The extent to which test designer & programmer are separated or linked depends on testing level and the context.

#	Programmer / Designer	Tester
1	Tests designed by designers are more oriented towards structural testing and are limited to its limitations.	With knowledge about internal test design, the tester can eliminate useless tests, optimize & do an efficient test design.
2	Likely to be biased.	Tests designed by independent testers are bias-free.
3	Tries to do the job in simplest & cleanest way, trying to reduce the complexity.	Tester needs to be suspicious, uncompromising, hostile and obsessed with destroying program.

Dichotomies

Dichotomies contd..

4. Modularity (Design) vs Efficiency

1. system and test design can both be modular.
2. A module implies a size, an internal structure and an interface, Or, in other words.
3. A module (well defined discrete component of a system) consists of internal complexity & interface complexity and has a size.

Dichotomies

#	Modularity	Efficiency
1	Smaller the component easier to understand.	Implies more number of components & hence more # of interfaces increase complexity & reduce efficiency (=> more bugs likely)
2	Small components/modules are repeatable independently with less rework (to check if a bug is fixed).	Higher efficiency at module level, when a bug occurs with small components.
3	Microscopic test cases need individual setups with data, systems & the software. Hence can have bugs.	More # of test cases implies higher possibility of bugs in test cases. Implies more rework and hence less efficiency with microscopic test cases
4	Easier to design large modules & smaller interfaces at a higher level.	Less complex & efficient. (Design may not be enough to understand and implement. It may have to be broken down to implementation level.)

So:

- ❑ Optimize the size & balance internal & interface complexity to increase efficiency
- ❑ Optimize the test design by setting the scopes of tests & group of tests (modules) to minimize cost of test design, debugging, execution & organizing – without compromising effectiveness.

Dichotomies

Dichotomies contd..

5. Programming in **SMALL** Vs programming in **BIG**

- ✓ Impact on the development environment due to the volume of customer requirements.

#	Small	Big
1	More efficiently done by informal, intuitive means and lack of formality – if it's done by 1 or 2 persons for small & intelligent user population.	A large # of programmers & large # of components.
2	Done for e.g., for oneself, for one's office or for the institute.	Program size implies non-linear effects (on complexity, bugs, effort, rework quality).
3	Complete test coverage is easily done.	Acceptance level could be: Test coverage of 100% for unit tests and for overall tests $\geq 80\%$.

Dichotomies

6. Buyer Vs Builder (customer vs developer organization)

- * Buyer & Builder being the same (organization) **clouds accountability**.
- * Separate them to **make the accountability clear**, even if they are in the same organization.
- * The accountability increases **motivation for quality**.
- * The **roles of all parties** involved are:
 - * **Builder:**
 - * Designs for & is accountable to the **Buyer**.
 - * **Buyer:**
 - * Pays for the **system**.
 - * Hopes to get profits from the services to the **User**.
 - * **User:**
 - * Ultimate beneficiary of the **system**.
 - * Interests are guarded by the **Tester**.
 - * **Tester:**
 - * *Dedicated to the destruction of the s/w (builder)*
 - * Tests s/w in the interests of User/Operator.
 - * **Operator:**
 - * Lives with:
 - Mistakes of the Builder**
 - Oversights of Tester**
 - Murky specs of Buyer**
 - Complaints of User**

Consequences of Bugs

Consequences: (how bugs may affect users)

These range from mild to catastrophic on a 10 point scale.

- **Mild**
 - Aesthetic bug such as misspelled output or mal-aligned print-out.
- **Moderate**
 - Outputs are misleading or redundant impacting performance.
- **Annoying**
 - Systems behavior is dehumanizing for e.g. names are truncated/modified arbitrarily, bills for \$0.0 are sent.
 - Till the bugs are fixed operators must use unnatural command sequences to get proper response.
- **Disturbing**
 - Legitimate transactions refused.
 - For e.g. ATM machine may malfunction with ATM card / credit card.
- **Serious**
 - Losing track of transactions & transaction events. Hence accountability is lost.

Consequences of Bugs

Consequences contd ...

- **Very serious**
System does another transaction instead of requested e.g. Credit another account, convert withdrawals to deposits.
- **Extreme**
 - Frequent & Arbitrary - not sporadic & unusual.
- **Intolerable**
 - Long term unrecoverable corruption of the Data base.
(not easily discovered and may lead to system down.)
- **Catastrophic**
 - System fails and shuts down.
- **Infectious**
 - Corrupts other systems, even when it may not fail.